Hook management for commands*

Frank Mittelbach Phelype Oleinik

October 19, 2026

Contents

1 Introduction **1**

2 Restrictions and operational details **3**
2.1 Patching . . . 3
2.1.1 Timing . . . 4
2.2 Command copies . . . 4
2.3 Grouping . . . 4
2.4 Commands that look ahead . . . 4

3 Package author interface **5**
3.1 Arguments and redefining commands . . . 6

4 The Implementation **7**
4.1 Execution plan . . . 7
4.2 Variables . . . 7
4.3 Patching or delaying . . . 8
4.4 Patching commands . . . 10
4.4.1 Patching by expansion and redefinition . . . 11
4.4.2 Patching by retokenization . . . 20
4.5 Messages . . . 26

Index **27**

1 Introduction

This file implements generic hooks for (arbitrary) commands. In theory every command \⟨*name*⟩ offers now two associated hooks to which code can be added using `\AddToHook`,[1] `\AddToHookNext`, `\AddToHookWithArguments`, and `\AddToHookNextWithArguments`.[2]

*This file has version v1.0n dated 2026-06-01, © LaTeX Project.

[1] In this documentation, when something is being said about `\AddToHook`, the same will be valid for `\AddToHookWithArguments`, unless that particular paragraph is highlighting the differences between both. The same is true for the other hook-related functions and their `...WithArguments` counterparts.

[2] In practice this is not supported for all types of commands, see section 2.4 for the restrictions that apply and what happens if one tries to use this with commands for which this is not supported.

However, this is only true "in theory". In practice there are a number of restrictions that makes it impossible to use such generic command hooks in a number of cases, so please read all of section 2 to understand what may prevent you from using them successfully.

The generic command hooks are:

cmd/⟨*name*⟩/before This hook is executed at the very start of the command, right after its arguments (if any) are parsed. The hook ⟨*code*⟩ runs in the command inside a call to `\UseHookWithArguments`. Any code added to this hook using `\AddToHookWithArguments` or `\AddToHookNextWithArguments` can access the command's arguments using `#1`, `#2`, etc., up to the number of arguments of the command. If `\AddToHook` or `\AddToHookNext` are used, the arguments cannot be accessed (see the lthooks documentation[3] on hooks with arguments).

cmd/⟨*name*⟩/after This hook is similar to `cmd/`⟨*name*⟩`/before`, but it is executed at the very end of the command body. This hook is implemented as a reversed hook.

The hooks are not physically present before `\begin{document}`[4] (i.e., using a command in the preamble will never execute the hook) and if nobody has declared any code for them, then they are not added to the command code ever. For example, if we have the following definition

```
\newcommand\foo[2]{Code #1 for #2!}
```

then executing `\foo{A}{B}` will simply run `Code␣A␣for␣B!` as it was always the case. However, if somebody, somewhere (e.g., in a package) adds

```
\AddToHook{cmd/foo/before}{<before code>}
```

then, after `\begin{document}` the definition of `\foo` will be:

```
\renewcommand\foo[2]{%
  \UseHookWithArguments{cmd/foo/before}{2}{#1}{#2}%
  Code #1 for #2!}
```

and similarly `\AddToHook{cmd/foo/after}{<after code>}` alters the definition to

```
\renewcommand\foo[2]{%
  Code #1 for #2!%
  \UseHookWithArguments{cmd/foo/after}{2}{#1}{#2}}
```

In other words, the mechanism is similar to what etoolbox offers with `\pretocmd` and `\apptocmd` with the important differences

- that code can be prepended or appended (i.e., added to the hooks) even if the command itself is not (yet) defined, because the defining package has not been loaded at this point;
- and that by using the hook management interface it is now possible to define how the code chunks added in these places are ordered, if different packages want to add code at these points.

[3] `texdoc lthooks-doc`

[4] More specifically, they are inserted in the commands after the `begindocument` hook, so they are also not present while LaTeX is reading the `.aux` file.

2 Restrictions and operational details

Adding arbitrary material to commands is tricky because most of the time we do not know what the macro expects as arguments when expanding and TEX doesn't have a reliable way to see that, so some guesswork has to be employed.

We can do this in most cases when commands are defined using `\NewDocumentCommand` or `\newcommand` (with a few exceptions). For commands defined with `\def` the situation is less good. Common cases where the command hooks will not work are:

- Commands that use special catcode settings within their definition. In that case it is usually not possible to augment the definition (see 2.1).
- If a command is defined while `\ExplSyntaxOn` is in force **and** the command contains `~` characters to represent spaces, then it can't be patched to include the command hooks. In fact in some very special circumstances you might even get a low-level error rather than the information that the command can't be patched (see, for example, https://github.com/latex3/latex2e/issues/1430.
- Commands that have arguments as far as the user is concerned (e.g., `\section` or `\caption`), but are defined in a way that these arguments are not read by the user level command but only later during the processing. In that case the `after` hook doesn't work at all. The before hook only works with `\AddToHook` but not with `\AddToHookWithArguments` because the arguments haven't been read at that point where the hook is patched in. See section 2.4.
- Adding a specific generic command hook is only attempted once per command, thus after redefining a command such hooks will no longer be there and will also not being re-added, see section 2.1.1.

All this means that you have to have a good understanding of how commands are defined when you attempt to make use of such hooks and something goes wrong. What can help in that case is to turn on `\DebugHooksOn` in which case you get much more (low-level) details on why something fails and what was tried to enable the hooks.

2.1 Patching

The code here tries to find out if a command was defined with `\newcommand` or `\DeclareRobustCommand` or `\NewDocumentCommand`, and if so it *assumes* that the argument specification of the command is as expected (which is not fail-proof, if someone redefines the internals of these commands in devious ways, but is a reasonable assumption).

If the command is one of the defined types, the code here does a sandboxed expansion of the command such that it can be redefined again exactly as before, but with the hook code added.

If however the command is not a known type (it was defined with `\def`, for example), then the code uses an approach similar to etoolbox's `\patchcmd` to retokenize the command with the hook code in place. This procedure, however, is more likely to fail if the catcode settings are not the same as the ones at the time of command's definition, so not always adding a hook to a command will work.

2.1.1 Timing

When `\AddToHook` (or its expl3 equivalent) is called with a generic `cmd` hook, say, `cmd/foo/before`, for the first time (that is, no code was added to that same hook before), in the preamble of a document, it will store a patch instruction for that command until `\begin{document}`, and only then all the commands which had hooks added will be patched in one go. That means that no command in the preamble will have hooks patched into them.

At `\begin{document}` all the delayed patches will be executed, and if the command doesn't exist the code is still added to the hook, but it will not be executed. After `\begin{document}`, when `\AddToHook` is called with a generic `cmd` hook the first time, the command will be immediately patched to include the hook, and if it doesn't exist or if it can't be patched for any reason, an error is thrown; if `\AddToHook` was already used in the preamble no new patching is attempted.

This has the consequence that a command defined or redefined after `\begin{document}` only uses generic `cmd` hook code if `\AddToHook` is called for the first time after the definition is made, or if the command explicitly uses the generic hook in its definition by declaring it with `\NewHookPair` adding `\UseHook` as part of the code.[5]

2.2 Command copies

Once a hook has been added to a command, it will be present in any copies of that command which are made. For example, if in the preamble we have

```
\NewDocumentCommand\foo{}{}
\AddToHook{cmd/foo/after}{1}%
```

then after `\begin{document}`, any use of `\NewCommandCopy` will include the hook in the copy, for example

```
\NewCommandCopy\baz\foo
```

would include the hook in this copied command.

2.3 Grouping

Command hooks are intended to be added to document commands, which are typically defined globally. As such, adding a hook is a global operation, even if the command was previously only locally-defined. Addition of material to hooks is also global.

2.4 Commands that look ahead

Some commands are defined in different "steps" and they look ahead in the input stream to find more arguments. If you try to add some code to the `cmd/`⟨*name*⟩`/after` hook of such command, it will not work, and it is not possible to detect that programmatically, so the user has to know (or find out) which commands can or cannot have hooks attached to them.

One good example is the `\section` command. You can add something to the `cmd/section/before` hook (but only with `\AddToHook` not `\AddToHookWithArguments`), but if you try to add anything to the `cmd/section/after` hook, `\section` will no longer

[5] We might change this behavior in the main document slightly after gaining some usage experience.

work at all. That happens because the `\section` macro takes no argument, but instead calls a few internal LaTeX macros to look for the optional and mandatory arguments. By adding code to the `cmd/section/after` hook, you get in the way of that scanning.

In such a case, where it is known that a specific generic command hook does not work if code is added to it, the package author can add a `\DisableGenericHook`[6] declaration to prevent this from happening in user documents and thereby avoiding obscure errors.

3 Package author interface

The `cmd` hooks are, by default, available for all commands that can be patched to add the hooks. For some commands, however, the very beginning or the very end of the code is not the best place to put the hooks, for example, if the command looks ahead for arguments (see section 2.4).

If you are a package author and you want to add the hooks to your own commands in the proper position you can define the command and manually add the `\UseHookWithArguments` calls inside the command in the proper positions, and manually define the hooks with `\NewHookWithArguments` or `\NewReversedHookWithArguments`. When the hooks are explicitly defined, patching is not attempted so you can make sure your command works properly. For example, an (admittedly not really useful) command that typesets its contents in a framed box with width optionally given in parentheses:

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{\parbox{#1}{#2}}}
```

If you try that definition, then add some code after it with

```
\AddToHook{cmd/fancybox/after}{<code>}
```

and then use the `\fancybox` command you will see that it will be completely broken, because the hook will get executed in the middle of parsing for optional `(...)` argument.

If, on the other hand, you want to add hooks to your command you can do something like:

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{%
              \UseHookWithArguments{cmd/fancybox/before}{2}{#1}{#2}%
              \parbox{#1}{#2}%
              \UseHookWithArguments{cmd/fancybox/after}{2}{#1}{#2}}}
\NewHookWithArguments{cmd/fancybox/before}{2}
\NewReversedHookWithArguments{cmd/fancybox/after}{2}
```

then the hooks will be executed where they should and no patching will be attempted. It is important that the hooks are declared with `\NewHookWithArguments` or `\NewReversedHookWithArguments`, otherwise the command hook code will try to patch the command. Note also that the call to `\UseHookWithArguments{cmd/fancybox/before}` does not need to be in the definition of `\fancybox`, but anywhere it makes sense to insert it (in this case in the internal `\@fancybox`).

[6] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

Alternatively, if for whatever reason your command does not support the generic hooks provided here, you can disable a hook with `\DisableGenericHook`[7], so that when someone tries to add code to it they will get an error. Or if you don't want the error, you can simply declare the hook with `\NewHook` and never use it.

The above approach is useful for really complex commands where for one or the other reason the hooks can't be placed at the very beginning and end of the command body and some hand-crafting is needed. However, in the example above the real (and in fact only) issue is the cascading argument parsing in the style developed long ago in LaTeX 2.09. Thus, a much simpler solution for this case is to replace it with the modern `\NewDocumentCommand` syntax and define the command as follows:

```
\DeclareDocumentCommand\fancybox{D(){5cm}m}{\fbox{\parbox{#1}{#2}}}
```

If you do that then both hooks automatically work and are patched into the right places.

3.1 Arguments and redefining commands

The code in ltcmdhooks does its best to find out how many arguments a given command has, and to insert the appropriate call to `\UseHookWithArguments`, so that the arguments seen by the hook are exactly those grabbed by the command (the hook, after all, is a macro call, so the arguments have to be placed in the right order, or they won't match).

When using the package writer interface, as discussed in section 3, to change the position of the hooks in your commands, you are also free to change how the hook code in your command sees its arguments. When a `cmd` hook is declared with `\NewHook` (or `\NewHookWithArguments` or other variations of that), it loses its "generic" nature and works as a regular hook. This means that you may choose to declare it without arguments regardless if the command takes arguments or not, or declare it with arguments, even if the command takes none.

However, this flexibility should not be abused. When using a nonstandard configuration for the hook arguments, think reasonably: a user will expect that the argument `#1` in the hook corresponds to the argument's first argument, and so on. Any other configuration is likely to cause confusion and, if used, will have to be well documented.

This flexibility, however, allows you to "correct" the arguments for the hooks. For example, LaTeX's `\refstepcounter` has a single argument, the name of the counter. The cleveref package adds an optional argument to `\refstepcounter`, making the name of the counter argument `#2`. If the author of cleveref wanted, for whatever reason, to add hooks to `\refstepcounter`, to preserve compatibility he could write something along the lines of:

```
\NewHookWithArguments{cmd/refstepcounter/before}{1}
\renewcommand\refstepcounter[2][<default>]{%
  \UseHookWithArguments{cmd/refstepcounter/before}{1}{#2}%
  <code for \refstepcounter>}
```

so that the mandatory argument, which is arg `#2` in the definition, would still be seen as `#1` in the hook code.

Another possibility would be to place the optional argument as the second argument for the hook, so that people looking for it would be able to use it. In either case, it would have to be well documented to cause as little confusion as possible.

[7] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

4 The Implementation

4.1 Execution plan

To add `before` and `after` hooks to a command we will need to peek into the definition of a command, which is always a tricky thing to do. Some cases are easy because we know how the command was defined, so we can assume how its ⟨*parameter text*⟩ looks like (for example a command defined with `\newcommand` may have an optional argument followed by a run of mandatory arguments), so we can just expand that command and make it grab `#1`, `#2`, etc. as arguments and define it all back with the hooks added.

Life's usually not that easy, so with some commands we can't do that (a `#1` might as well be $\texttt{\#}_{12}1_{12}$ instead of the expected $\texttt{\#}_{6}1_{12}$, for example) so we need to resort to "patching" the command: read its `\meaning`, and tokenize it again with `\scantokens` and hope for the best.

So the overall plan is:

1. Check if a command is of a known type (that is, defined with `\newcommand`[8], `\DeclareRobustCommand`, or `\New(Expandable)DocumentCommand`), and if is, take appropriate action.
2. If the command is not a known type, we'll check if the command can be patched. Two things will prevent a command from being patched: if it was defined in a nonstandard catcode setting, or if it is an internal expl3 command with `__`⟨*module*⟩ in its name, in which case we refuse to patch.
3. If the command was defined in nonstandard catcode settings, we will try a few standard ones to try our best to carry out the pathing. If this doesn't help either, the code will give up and throw an error.

```
⟨@@=hook⟩

⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2021/06/01}{ltcmdhooks}
⟨latexrelease⟩                        {The~hook~management~system~for~commands}
```

4.2 Variables

`\g_hook_patch_action_list_tl`

Pairs of `\if<cmd>..\patch<cmd>` to be used with `\robust@command@act` when looking for a known patching rule. This token list is exposed because we see some future applications (with very specialized packages, such as etoolbox that may want to extend the pairs processed. It is not meant for general use which is why it is not documented in the interface documentation above.

```
\tl_new:N \g_hook_patch_action_list_tl
```

(*End of definition for* `\g_hook_patch_action_list_tl`.)

`\l__hook_patch_num_args_int`

The number of arguments in a macro being patched.

```
\int_new:N \l__hook_patch_num_args_int
```

(*End of definition for* `\l__hook_patch_num_args_int`.)

[8] It's not always possible to reliably detect this case because a command defined with no optional argument is indistinguishable from a `\def`ed command.

\l__hook_patch_prefixes_tl
\l__hook_param_text_tl
\l__hook_replace_text_tl

The prefixes and parameters of the definition for the macro being patched.

```
\tl_new:N \l__hook_patch_prefixes_tl
\tl_new:N \l__hook_param_text_tl
\tl_new:N \l__hook_replace_text_tl
```

(*End of definition for* `\l__hook_patch_prefixes_tl`, `\l__hook_param_text_tl`, *and* `\l__hook_replace_text_tl`.)

\c__hook_hash_tl
\c__hook_hashes_tl

Two constant token lists that contain one and two parameter tokens.

```
\tl_const:Nn \c__hook_hash_tl { # }
\tl_const:Nn \c__hook_hashes_tl { # # }
```

(*End of definition for* `\c__hook_hash_tl` *and* `\c__hook_hashes_tl`.)

__hook_exp_not:NN
__hook_def_cmd:w

Two temporary macros that change depending on the macro being patched.

```
\cs_new_eq:NN \__hook_exp_not:NN ?
\cs_new_eq:NN \__hook_def_cmd:w ?
```

(*End of definition for* `\__hook_exp_not:NN` *and* `\__hook_def_cmd:w`.)

\q__hook_recursion_tail
\q__hook_recursion_stop

Internal quarks for recursion: they can't appear in any macro being patched.

```
\quark_new:N \q__hook_recursion_tail
\quark_new:N \q__hook_recursion_stop
```

(*End of definition for* `\q__hook_recursion_tail` *and* `\q__hook_recursion_stop`.)

\g__hook_delayed_patches_prop

A list containing the patches delayed to `\begin{document}`, so that patching is not attempted twice.

```
\prop_new:N \g__hook_delayed_patches_prop
```

(*End of definition for* `\g__hook_delayed_patches_prop`.)

__hook_patch_debug:e

A helper for patching debug info.

```
\cs_new_protected:Npn \__hook_patch_debug:e #1
  { \__hook_debug:n { \iow_term:e { [lthooks]~#1 } } }
```

(*End of definition for* `\__hook_patch_debug:e`.)

4.3 Patching or delaying

Before `\begin{document}` all patching is delayed.

__hook_try_put_cmd_hook:n
__hook_try_put_cmd_hook:w

This function is called from within `\AddToHook`, when code is first added to a generic `cmd` hook. If it is called within in the preamble, it delays the action until `\begin{document}`; otherwise it tries to update the hook.

```
⟨latexrelease⟩\IncludeInRelease{2024/12/22}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                    {Don't~define~command}
\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
    #1 / #2 / #3 / #4 \s__hook_mark #5
  {
    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
```

`\__hook_patch_cmd_or_delay:Nnn` expects the command to be patched as its first argument so we need to construct it from its name (`#2`). However, at this moment it may not exist yet, so using `\cs:w` would incorrectly turn it from "undefined" into `\relax`. We therefore use the following curious construction: we start a group and expand out of it to call `\cs:w`. If the command is now changed to `\relax` the `\group_end:` will undo that change, but the token is nevertheless there to be consumed by `\__hook_patch_cmd_or_delay:Nnn`.

```
    \group_begin:
      \exp_after:wN
    \group_end:
      \exp_after:wN
    \__hook_patch_cmd_or_delay:Nnn
      \cs:w #2\cs_end:
      {#2} {#3}
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
⟨latexrelease⟩  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
⟨latexrelease⟩    #1 / #2 / #3 / #4 \s__hook_mark #5
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
⟨latexrelease⟩    \exp_args:Nc \__hook_patch_cmd_or_delay:Nnn {#2} {#2} {#3}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
⟨latexrelease⟩  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
⟨latexrelease⟩    #1 / #2 / #3 / #4 \s__hook_mark #5
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
⟨latexrelease⟩    \str_case:nnTF {#3}
⟨latexrelease⟩        { { before } { } { after } { } }
⟨latexrelease⟩      { \exp_args:Nc \__hook_patch_cmd_or_delay:Nnn {#2} {#2} {#3} }
⟨latexrelease⟩      { \msg_error:nnnn { hooks } { wrong-cmd-hook } {#2} {#3} }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_put_cmd_hook:n` *and* `\__hook_try_put_cmd_hook:w`.)

`\__hook_patch_cmd_or_delay:Nnn`
`\__hook_cmd_begindocument_code:`

In the preamble, `\__hook_patch_cmd_or_delay:Nnn` just adds the patch instruction to a property list to be executed later.

```
\cs_new_protected:Npn \__hook_patch_cmd_or_delay:Nnn #1 #2 #3
  {
    \__hook_debug:n { \iow_term:n { ->~Add~generic~cmd~hook~for~#2~(#3). } }
    \__hook_debug:n
      { \iow_term:n { !~In~the~preamble:~delaying. } }
    \prop_gput:Nnn \g__hook_delayed_patches_prop { #2 / #3 }
      { \__hook_cmd_try_patch:nn {#2} {#3} }
```

```
}
```

The delayed patches are added to a property list to prevent duplication, and the code stored in the property list for each key is executed. The function `\__hook_patch_cmd_or_delay:Nnn` is also redefined to be `\__hook_patch_command:Nnn` so that no further delaying is attempted.

```
\cs_new_protected:Npn \__hook_cmd_begindocument_code:
  {
    \cs_gset_eq:NN \__hook_patch_cmd_or_delay:Nnn \__hook_patch_command:Nnn
    \prop_map_function:NN \g__hook_delayed_patches_prop { \use_ii:nn }
    \prop_gclear:N \g__hook_delayed_patches_prop
    \cs_undefine:N \__hook_cmd_begindocument_code:
  }
\g@addto@macro \@kernel@after@begindocument
  { \__hook_cmd_begindocument_code: }
```

(*End of definition for* `\__hook_patch_cmd_or_delay:Nnn` *and* `\__hook_cmd_begindocument_code:`.)

`\__hook_cmd_try_patch:nn`

At `\begin{document}` tries patching the command if the hook was not manually created in the meantime. If the document does not exist, no error is raised here as it may hook into a package that wasn't loaded. Hooks added to commands in the document body still raise an error if the command is not defined.

```
\cs_new_protected:Npn \__hook_cmd_try_patch:nn #1 #2
  {
    \__hook_debug:n
      { \iow_term:e { ->~\string\begin{document}~try~cmd / #1 / #2. } }
    \__hook_if_declared:nTF { cmd / #1 / #2 }
      {
        \__hook_debug:n
          { \iow_term:n { .->~Giving~up:~hook~already~created. } }
      }
      {
        \cs_if_exist:cT {#1}
          { \exp_args:Nc \__hook_patch_command:Nnn {#1} {#1} {#2} }
      }
  }
```

(*End of definition for* `\__hook_cmd_try_patch:nn`.)

4.4 Patching commands

`\__hook_patch_command:Nnn`
`\__hook_patch_check:NNnn`
`\__hook_if_public_command:NTF`
`\__hook_if_public_command:w`

`\__hook_patch_command:Nnn` will do some sanity checks on the argument to detect if it is possible to add hooks to the command, and raises an error otherwise. If the command can contain hooks, then it uses `\robust@command@act` to find out what type is the command, and patch it accordingly.

```
\cs_new_protected:Npn \__hook_patch_command:Nnn #1 #2 #3
  {
    \__hook_patch_debug:e { analyzing~'\token_to_str:N #1' }
    \__hook_patch_debug:e { \token_to_str:N #1 = \token_to_meaning:N #1 }
    \__hook_patch_check:NNnn \cs_if_exist:NTF #1 { undef }
      {
        \__hook_patch_debug:e { ++~control~sequence~is~defined }
        \__hook_patch_check:NNnn \token_if_macro:NTF #1 { macro }
          {
```

```
            \__hook_patch_debug:e { ++~control~sequence~is~a~macro }
            \__hook_patch_check:NNnn \__hook_if_public_command:NTF #1 { expl3 }
              {
                \__hook_patch_debug:e { ++~macro~is~not~private }
                \robust@command@act
                  \g_hook_patch_action_list_tl #1
                  \__hook_retokenize_patch:Nnn { #1 {#2} {#3} }
              }
          }
      }
  }
```

And here's the auxiliary used above:

```
\cs_new_protected:Npn \__hook_patch_check:NNnn #1 #2 #3 #4
  {
    #1 #2 {#4}
      {
        \msg_error:nnee { hooks } { cant-patch }
          { \token_to_str:N #2 } {#3}
      }
  }
```

and a conditional `\__hook_if_public_command:NTF` to check if a command has `__` in its name (no other checking is performed). Primitives with `:D` in their name could be included here, but they are already discarded in the `\token_if_macro:NTF` test above.

```
\use:e
  {
    \prg_new_protected_conditional:Npnn
        \exp_not:N \__hook_if_public_command:N #1 { TF }
      {
        \exp_not:N \exp_last_unbraced:Nf
          \exp_not:N \__hook_if_public_command:w
            { \exp_not:N \cs_to_str:N #1 }
          \tl_to_str:n { _ _ } \s__hook_mark
      }
  }
\exp_last_unbraced:NNNNo
\cs_new_protected:Npn \__hook_if_public_command:w
    #1 \tl_to_str:n { _ _ } #2 \s__hook_mark
  {
    \tl_if_empty:nTF {#2}
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_patch_command:Nnn` *and others.*)

4.4.1 Patching by expansion and redefinition

`\g_hook_patch_action_list_tl`

This is the list of known command types and the function that patches the command hooks into them. The conditionals are taken from `\ShowCommand`, `\NewCommandCopy` and `\__kernel_cmd_if_xparse:NTF` defined in `ltcmd`.

```
\tl_gset:Nn \g_hook_patch_action_list_tl
  {
    { \@if@DeclareRobustCommand \__hook_patch_DeclareRobustCommand:Nnn }
```

```
    { \@if@newcommand \__hook_patch_newcommand:Nnn }
    { \__kernel_cmd_if_xparse:NTF \__hook_cmd_patch_xparse:Nnn }
  }
```

(*End of definition for* `\g_hook_patch_action_list_tl`.)

`\__hook_patch_DeclareRobustCommand:Nnn`

At this point we know that the commands can be patched by expanding then redefining. These are the cases of commands defined with `\newcommand` with an optional argument or with `\DeclareRobustCommand`.

With `\__hook_patch_DeclareRobustCommand:Nnn` we check if the command has an optional argument (with a test counter-intuitively called `\@if@newcommand`; also make sure the command doesn't take args by calling `\robust@command@chk@safe`). If so, we pass the patching action to `\__hook_patch_newcommand:Nnn`, otherwise we call the patching engine `\__hook_patch_expand_redefine:NNnn` with a `\c_false_bool` to indicate that there is no optional argument.

```
\cs_new_protected:Npn \__hook_patch_DeclareRobustCommand:Nnn #1
  {
    \exp_args:Nc \__hook_patch_DeclareRobustCommand_aux:Nnn
      { \cs_to_str:N #1 ~ }
  }
\cs_new_protected:Npn \__hook_patch_DeclareRobustCommand_aux:Nnn #1
  {
    \robust@command@chk@safe #1
      { \@if@newcommand #1 }
      { \use_ii:nn }
        { \__hook_patch_newcommand:Nnn }
        { \__hook_patch_expand_redefine:NNnn \c_false_bool }
          #1
  }
```

(*End of definition for* `\__hook_patch_DeclareRobustCommand:Nnn`.)

`\__hook_patch_newcommand:Nnn`

If the command was defined with `\newcommand` and an optional argument, call the patching engine with a `\c_true_bool` to flag the presence of an optional argument, and with `\\command` to patch the actual code for `\command`.

```
\cs_new_protected:Npn \__hook_patch_newcommand:Nnn #1
  {
    \exp_args:NNc \__hook_patch_expand_redefine:NNnn \c_true_bool
      { \c_backslash_str \cs_to_str:N #1 }
  }
```

(*End of definition for* `\__hook_patch_newcommand:Nnn`.)

`\__hook_cmd_patch_xparse:Nnn`

And for commands defined by the xparse commands use this for patching:

```
\cs_new_protected:Npn \__hook_cmd_patch_xparse:Nnn #1
  {
    \exp_args:NNc \__hook_patch_expand_redefine:NNnn \c_false_bool
      { \cs_to_str:N #1 ~ code }
  }
```

(*End of definition for* `\__hook_cmd_patch_xparse:Nnn`.)

__hook_patch_expand_redefine:NNnn
__hook_redefine_with_hooks:Nnnn
__hook_make_prefixes:w

Now the real action begins. Here we have in `#1` a boolean indicating if the command has a leading `[...]`-delimited argument, in `#2` the command control sequence, in `#3` the name of the command (note that `#1` $\neq$ `\csname#2\endcsname` at this point!), and in `#4` the hook position, either `before` or `after`.

Patching with expansion+redefinition is trickier than it looks like at first glance. Suppose the simple definition:

```
\def\foo#1{#1##2}
```

When defined, its ⟨*replacement text*⟩ will be a token list containing:

out_param `1`, *mac_param* `#`, *character* `2`

Then, after expanding `\foo{##1}` (here `##` denotes a single $\#_6$) we end up with a token list with *out_param* `1` replaced:

mac_param `#`, *character* `1`, *mac_param* `#`, *character* `2`

that is, the definition would be:

```
\def\foo#1{#1#2}
```

which obviously fails, because the original input in the definition was `##` but TEX reduced that to a single parameter token $\#_6$ when carrying out the definition. That leaves no room for a clever solution with (say) `\unexpanded`, because anything that would double the second $\#_6$, would also (incorrectly) double the first, so there's not much to do other than a manual solution.

There are three cases we can distinguish to make things hopefully faster on simpler cases:

1. a macro with no parameters;
2. a macro with no parameter tokens in its definition;
3. a macro with parameters *and* parameter tokens.

The first case is trivial: if the macro has no parameters, we can just use `\unexpanded` around it, and if there is a parameter token in it, it is handled correctly (the macro can be treated as a `tl` variable).

The second case requires looking at the ⟨*replacement text*⟩ of the macro to see if it has a parameter token in there. If it does not, then there is no worry, and the macro can be redefined normally (without `\unexpanded`).

The third case, as usual, is the devious one. Here we'll have to loop through the definition token by token, and double every parameter token, so that this case can be handled like the previous one.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_patch_expand_redefine:NNnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_patch_expand_redefine:NNnn #1 #2 #3 #4
  {
    \__hook_patch_debug:e { ++~command~can~be~patched~without~rescanning }
```

We'll start by counting the number of arguments in the command by counting the number of characters in the `\cs_parameter_spec:N` of the macro, divided by two, and subtracting one if the command has an optional argument (that is, an extra `[]` in its ⟨*parameter text*⟩).

```
\int_set:Nn \l__hook_patch_num_args_int
  {
    \exp_args:Nf \str_count:n { \__kernel_cs_parameter_spec:N #2 } / 2
    \bool_if:NT #1 { -1 }
  }
```

Now build two token lists:

`\l__hook_param_text_tl` will contain the ⟨*parameter text*⟩ to be used when redefining the macro. It should be identical to the ⟨*parameter text*⟩ used when originally defining that macro.

`\l__hook_replace_text_tl` will contain braced pairs of `\c__hook_hashes_tl`⟨*num*⟩ to feed to the macro when expanded. This token list as well as the previous will have the first item surrounded by `[...]` in the case of an optional argument.

The use of `\c__hook_hashes_tl` here is to differentiate actual parameters in the macro from parameter tokens in the original definition of the macro. Later on, `\c__hook_hashes_tl` is either replaced by actual parameter tokens, or expanded into them.

```
\int_compare:nNnTF { \l__hook_patch_num_args_int } > { \c_zero_int }
  {
```

We'll first check if the command has any parameter token in its definition (feeding it empty arguments), and set `\__hook_exp_not:n` accordingly. `\__hook_exp_not:n` will be used later to either leave `\c__hook_hashes_tl` or expand it, and also to remember the result of `\__hook_if_has_hash:nTF` to avoid testing twice (the test can be rather slow).

```
    \tl_set:Ne \l__hook_tmpa_tl { \bool_if:NTF #1 { [ ] } { { } } }
    \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
      { \tl_put_right:Nn \l__hook_tmpa_tl { { } } }
    \exp_args:NNo \exp_args:No \__hook_if_has_hash:nTF
        { \exp_after:wN #2 \l__hook_tmpa_tl }
      { \cs_set_eq:NN \__hook_exp_not:n \exp_not:n }
      { \cs_set_eq:NN \__hook_exp_not:n \use:n }
    \cs_set_protected:Npn \__hook_tmp:w ##1 ##2
      {
        ##1 \l__hook_param_text_tl   { \use:n ##2 }
        ##1 \l__hook_replace_text_tl { \__hook_exp_not:n {##2} }
      }
```

Here we'll conditionally add `[...]` around the first parameter:

```
    \bool_if:NTF #1
      { \__hook_tmp:w \tl_set:Ne { [ \c__hook_hash_tl 1 ] } }
      { \__hook_tmp:w \tl_set:Ne { { \c__hook_hash_tl 1 } } }
```

Then, for every parameter from the second, just add it normally:

```
    \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
      { \__hook_tmp:w \tl_put_right:Ne { { \c__hook_hash_tl ##1 } } }
```

Now, if the command has any parameter token in its definition (then `\__hook_exp_not:n` is `\exp_not:n`), call `\__hook_double_hashes:n` to double them, and replace every `\c__hook_hashes_tl` by `#`:

```
      \tl_set:Ne \l__hook_replace_text_tl
        { \exp_not:N #2 \exp_not:V \l__hook_replace_text_tl }
      \tl_set:Ne \l__hook_replace_text_tl
        {
          \token_if_eq_meaning:NNTF \__hook_exp_not:n \exp_not:n
            { \exp_args:NNV \exp_args:No \__hook_double_hashes:n }
            { \exp_args:NV \exp_not:o }
                \l__hook_replace_text_tl
        }
```

And now, set a few auxiliaries for the case that the macro has parameters, so it won't be passed through `\unexpanded` (twice):

```
      \cs_set_eq:NN \__hook_def_cmd:w \tex_gdef:D
      \cs_set_eq:NN \__hook_exp_not:NN \prg_do_nothing:
    }
    {
```

In the case the macro has no parameters, we'll treat it as a token list and things are much simpler (expansion control looks a bit complicated, but it's just a pair of `\exp_not:N` preventing another `\exp_not:n` from expanding):

```
      \tl_clear:N \l__hook_param_text_tl
      \tl_set_eq:NN \l__hook_replace_text_tl #2
      \cs_set_eq:NN \__hook_def_cmd:w \tex_xdef:D
      \cs_set:Npn \__hook_exp_not:NN ##1 { \exp_not:N ##1 \exp_not:N }
    }
```

Before redefining, we need to also get the prefixes used when defining the command. Here we ensure that the `\escapechar` is printable, otherwise a macro defined with prefixes `\protected \long` will have it `\meaning` printed as `protectedlong`, making life unnecessarily complicated. Here the `\escapechar` is changed to `/`, then we loop between pairs of `/.../` extracting the prefixes.

```
  \group_begin:
    \int_set:Nn \tex_escapechar:D { `\/ }
    \use:e
      {
  \group_end:
  \tl_set:Ne \exp_not:N \l__hook_patch_prefixes_tl
    { \exp_not:N \__hook_make_prefixes:w \cs_prefix_spec:N #2 / / }
      }
```

Here we redefine the hook to have the right number of arguments. Disabling the hook, undefining the `parameter` token list then calling `\__hook_make_usable:nn` are enough to redefine the hook to the extent we want. Code stored in the hook and other metadata about it are not lost in the process.

```
  \__hook_disable:n { cmd / #3 / #4 }
  \cs_undefine:c { c__hook_cmd / #3 / #4_parameter_tl }
  \__hook_make_usable:nn { cmd / #3 / #4 } { \l__hook_patch_num_args_int }
```

Now call `\__hook_redefine_with_hooks:Nnnn` with the macro being redefined in `#1`, then `\UseHook{cmd/<name>/before}` in `#2` or `\UseHook{cmd/<name>/after}` in `#3` (one is always empty), and in `#4` the ⟨*replacement text*⟩ of the macro.

```
    \use:e
      {
        \__hook_redefine_with_hooks:Nnnn \exp_not:N #2
        \str_if_eq:nnTF {#4} { after }
          { \use_ii_i:nn }
          { \use:nn }
            { {
                \__hook_exp_not:NN \exp_not:N \UseHookWithArguments
                  { cmd / #3 / #4 } { \int_use:N \l__hook_patch_num_args_int }
                  \__hook_braced_parameter:n { cmd / #3 / #4 }
            } }
            { { } }
            { \__hook_exp_not:NN \exp_not:V \l__hook_replace_text_tl }
      }
```

Finally, update the hook code.

```
    \__hook_update_hook_code:n { cmd / #3 / #4 }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_patch_expand_redefine:NNnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_patch_expand_redefine:NNnn #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_patch_debug:e { ++~command~can~be~patched~without~rescanning }
⟨latexrelease⟩    \int_set:Nn \l__hook_patch_num_args_int
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Nf \str_count:n { \__kernel_cs_parameter_spec:N #2 } / 2
⟨latexrelease⟩        \bool_if:NT #1 { -1 }
⟨latexrelease⟩      }
⟨latexrelease⟩    \int_compare:nNnTF { \l__hook_patch_num_args_int } > { \c_zero_int }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_set:Nx \l__hook_tmpa_tl { \bool_if:NTF #1 { [ ] } { { } } }
⟨latexrelease⟩        \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
⟨latexrelease⟩          { \tl_put_right:Nn \l__hook_tmpa_tl { { } } }
⟨latexrelease⟩        \exp_args:NNo \exp_args:No \__hook_if_has_hash:nTF
⟨latexrelease⟩            { \exp_after:wN #2 \l__hook_tmpa_tl }
⟨latexrelease⟩          { \cs_set_eq:NN \__hook_exp_not:n \exp_not:n }
⟨latexrelease⟩          { \cs_set_eq:NN \__hook_exp_not:n \use:n }
⟨latexrelease⟩        \cs_set_protected:Npn \__hook_tmp:w ##1 ##2
⟨latexrelease⟩          {
⟨latexrelease⟩            ##1 \l__hook_param_text_tl   { \use:n ##2 }
⟨latexrelease⟩            ##1 \l__hook_replace_text_tl { \__hook_exp_not:n {##2} }
⟨latexrelease⟩          }
⟨latexrelease⟩        \bool_if:NTF #1
⟨latexrelease⟩          { \__hook_tmp:w \tl_set:Nx { [ \c__hook_hash_tl 1 ] } }
⟨latexrelease⟩          { \__hook_tmp:w \tl_set:Nx { { \c__hook_hash_tl 1 } } }
⟨latexrelease⟩        \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
⟨latexrelease⟩          { \__hook_tmp:w \tl_put_right:Nx { { \c__hook_hash_tl ##1 } } }
⟨latexrelease⟩        \tl_set:Nx \l__hook_replace_text_tl
⟨latexrelease⟩          { \exp_not:N #2 \exp_not:V \l__hook_replace_text_tl }
⟨latexrelease⟩        \tl_set:Nx \l__hook_replace_text_tl
⟨latexrelease⟩          {
⟨latexrelease⟩            \token_if_eq_meaning:NNTF \__hook_exp_not:n \exp_not:n
⟨latexrelease⟩              { \exp_args:NNV \exp_args:No \__hook_double_hashes:n }
⟨latexrelease⟩              { \exp_args:NV \exp_not:o }
```

```
⟨latexrelease⟩                  \l__hook_replace_text_tl
⟨latexrelease⟩          }
⟨latexrelease⟩        \cs_set_eq:NN \__hook_def_cmd:w \tex_gdef:D
⟨latexrelease⟩        \cs_set_eq:NN \__hook_exp_not:NN \prg_do_nothing:
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_clear:N \l__hook_param_text_tl
⟨latexrelease⟩        \tl_set_eq:NN \l__hook_replace_text_tl #2
⟨latexrelease⟩        \cs_set_eq:NN \__hook_def_cmd:w \tex_xdef:D
⟨latexrelease⟩        \cs_set:Npn \__hook_exp_not:NN ##1 { \exp_not:N ##1 \exp_not:N }
⟨latexrelease⟩      }
⟨latexrelease⟩    \group_begin:
⟨latexrelease⟩      \int_set:Nn \tex_escapechar:D { `\/ }
⟨latexrelease⟩      \use:x
⟨latexrelease⟩        {
⟨latexrelease⟩    \group_end:
⟨latexrelease⟩    \tl_set:Nx \exp_not:N \l__hook_patch_prefixes_tl
⟨latexrelease⟩      { \exp_not:N \__hook_make_prefixes:w \cs_prefix_spec:N #2 / / }
⟨latexrelease⟩        }
⟨latexrelease⟩    \use:x
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_redefine_with_hooks:Nnnn \exp_not:N #2
⟨latexrelease⟩        \str_if_eq:nnTF {#4} { after }
⟨latexrelease⟩          { \use_ii_i:nn }
⟨latexrelease⟩          { \use:nn }
⟨latexrelease⟩            { { \__hook_exp_not:NN \exp_not:N \UseHook { cmd / #3 / #4 } } }
⟨latexrelease⟩            { { } }
⟨latexrelease⟩            { \__hook_exp_not:NN \exp_not:V \l__hook_replace_text_tl }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

Now that all the needed tools are ready, without further ado we'll redefine the command. The definition uses the prefixes gathered in `\l__hook_patch_prefixes_tl`, a primitive `\__hook_def_cmd:w` (which is `\tex_gdef:D` or `\tex_xdef:D`) to avoid adding extra prefixes, and the ⟨*parameter text*⟩ from `\l__hook_param_text_tl`.

Then finally, in the body of the definition, we insert `#2`, which is `cmd/#1/before` or empty, `#4` which is the ⟨*replacement text*⟩, and `#3` which is `cmd/#1/after` or empty.

```
\cs_new_protected:Npn \__hook_redefine_with_hooks:Nnnn #1 #2 #3 #4
  {
    \l__hook_patch_prefixes_tl
      \exp_after:wN \__hook_def_cmd:w
        \exp_after:wN #1 \l__hook_param_text_tl
      { #2  #4  #3 }
  }
```

Here's the auxiliary that makes the prefix control sequences for the redefinition. Each item has to be `\tl_trim_spaces:n`'d because the last item (and not any other) has a trailing space.

```
\cs_new:Npn \__hook_make_prefixes:w / #1 /
  {
    \tl_if_empty:nF {#1}
      {
        \exp_not:c { tex_ \tl_trim_spaces:n {#1} :D }
```

```
        \__hook_make_prefixes:w /
      }
  }
```

(*End of definition for* `\__hook_patch_expand_redefine:NNnn`, `\__hook_redefine_with_hooks:Nnnn`, *and* `\__hook_make_prefixes:w`.)

Here are some auxiliaries for the contraption above.

`\__hook_if_has_hash_p:n`
`\__hook_if_has_hash:nTF`
`\__hook_if_has_hash:w`
`\__hook_if_has_hash_check:w`

`\__hook_if_has_hash:nTF` searches the token list `#1` for a catcode 6 token, and if any is found, it returns `true`, and `false` otherwise. The searching doesn't care about preserving groups or spaces: we can ignore those safely (braces are removed) so that searching is as fast as possible.

```
\prg_new_conditional:Npnn \__hook_if_has_hash:n #1 { TF }
  { \__hook_if_has_hash:w #1 ## \s__hook_mark }
\cs_new:Npn \__hook_if_has_hash:w #1
  {
    \tl_if_single_token:nTF {#1}
      {
        \token_if_eq_catcode:NNTF ## #1
          { \__hook_if_has_hash_check:w }
          { \__hook_if_has_hash:w }
      }
      { \__hook_if_has_hash:w #1 }
  }
\cs_new:Npn \__hook_if_has_hash_check:w #1 \s__hook_mark
  { \tl_if_empty:nTF {#1} { \prg_return_false: } { \prg_return_true: } }
```

(*End of definition for* `\__hook_if_has_hash:nTF`, `\__hook_if_has_hash:w`, *and* `\__hook_if_has_hash_check:w`.)

`\__hook_double_hashes:n`
`\__hook_double_hashes:w`
`\__hook_double_hashes_output:N`
`\__hook_double_hashes_output_aux:N`
`\__hook_double_hashes_stop:w`
`\__hook_double_hashes_group:n`
`\__hook_double_hashes_space:w`

`\__hook_double_hashes:n` loops through the token list `#1` and duplicates any catcode 6 token, and expands tokens `\ifx`-equal to `\c__hook_hashes_tl`, and leaves all other tokens `\notexpanded` with `\exp_not:N`. Unfortunately pairs of explicit catcode 1 and catcode 2 character tokens are normalized to $\{_1$ and $\}_1$ because it's not feasible to expandably detect the character code (*maybe* it could be done using something along the lines of https://tex.stackexchange.com/a/527538, but it's far too much work for close to zero benefit).

`\__hook_double_hashes:w` is the tail-recursive loop macro, that tests which of the three types of item is in the head of the token list.

```
\cs_new:Npn \__hook_double_hashes:n #1
  { \__hook_double_hashes:w #1 \q__hook_recursion_tail \q__hook_recursion_stop }
\cs_new:Npn \__hook_double_hashes:w #1 \q__hook_recursion_stop
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__hook_double_hashes_output:N }
      {
        \tl_if_head_is_group:nTF {#1}
          { \__hook_double_hashes_group:n }
          { \__hook_double_hashes_space:w }
      }
    #1 \q__hook_recursion_stop
  }
```

`\__hook_double_hashes_output:N` checks for the end of the token list, then checks if the token is `\c__hook_hashes_tl`, and if so just leaves it.

```
\cs_new:Npn \__hook_double_hashes_output:N #1
  {
    \if_meaning:w \q__hook_recursion_tail #1
      \__hook_double_hashes_stop:w
    \fi:
    \if:w ?
      \if_meaning:w \c__hook_hash_tl #1 ! \fi:
      \if_meaning:w \c__hook_hashes_tl #1 ! \fi:
          ?
    \else:
```

(this `\use_i:nnnn` uses `\fi:` and consumes `\use:n`, the whole `\if_catcode:w` block, and the `\exp_not:N`, leaving just `#1` which is `\c__hook_hashes_tl`.)

```
      \use_i:nnnn
    \fi:
    \use:n
      {
```

If `#1` is not `\c__hook_hashes_tl`, then check if its catcode is 6, and if so, leave it doubled in `\exp_not:n` and consume the following `\exp_not:N #1`.

```
        \__hook_double_hashes_output_aux:N #1
        \use_none:n
          { \exp_not:n { #1 #1 } }
      }
```

If both previous tests returned `false`, then leave the token unexpanded and resume the loop.

```
    \exp_not:N #1
    \__hook_double_hashes:w
  }
```

LuaTEX has a primitive equivalent to `#`, which is replaced in a dedicated path. That code is not needed for other engines, so we avoid the hit on performance by using an auxiliary.

```
\cs_new:Npn \__hook_double_hashes_output_aux:N #1
  {
    \if_catcode:w ## \exp_not:N #1
      \exp_after:wN \use_ii:nnnn
    \fi:
  }
\sys_if_engine_luatex:T
  {
    \cs_gset:Npn \__hook_double_hashes_output_aux:N #1
      {
        \if_catcode:w ## \exp_not:N #1
          \exp_after:wN \use_ii:nnnn
        \else:
          \if_meaning:w \tex_alignmark:D \tex_alignmark:D #1
            \exp_after:wN \exp_after:wN \exp_after:wN \use_ii:nnnn
          \fi:
        \fi:
      }
  }
\cs_new:Npn \__hook_double_hashes_stop:w #1 \q__hook_recursion_stop { \fi: }
```

Dealing with spaces and grouped tokens is trivial:

```
\cs_new:Npn \__hook_double_hashes_group:n #1
  { { \__hook_double_hashes:n {#1} } \__hook_double_hashes:w }
\exp_last_unbraced:NNo
\cs_new:Npn \__hook_double_hashes_space:w \c_space_tl
  { ~ \__hook_double_hashes:w }
```

(*End of definition for* `\__hook_double_hashes:n` *and others.*)

4.4.2 Patching by retokenization

At this point we've drained the possibilities of patching a command by expansion-and-redefinition, so we have to resort to patching by retokenizing the command. Patching by retokenization is done by getting the `\meaning` of the command, doing the necessary manipulations on the generated string, and the retokenizing that again by using `\scantokens`.

Patching by retokenization is definitely a riskier business, because it relies that the tokens printed by `\meaning` produce the exact same tokens as the ones in the original definition. That is, the catcode régime must be exactly(ish) the same, and there is no way of telling except by trial and error.

`\__hook_retokenize_patch:Nnn` This is the macro that will control the whole process. First we'll try out one final, rather trivial case, of a command with no arguments; that is, a token list. This case can be patched with the expand-and-redefine routine but it has to be the very last case tested for, because most (all?) robust commands start with a top-level macro with no arguments, so testing this first would short-circuit `\robust@command@act` and the top-level macros would be incorrectly patched. In that case, we just check if the `\cs_parameter_spec:N` is empty, and call `\__hook_patch_expand_redefine:NNnn`.

```
\cs_new_protected:Npn \__hook_retokenize_patch:Nnn #1 #2 #3
  {
    \str_if_eq:eeTF { \__kernel_cs_parameter_spec:N #1 } { }
      { \__hook_patch_expand_redefine:NNnn \c_false_bool #1 {#2} {#3} }
      {
        \__hook_patch_debug:e { ..~command~can~only~be~patched~by~rescanning }
```

Otherwise, we start the actual patching by retokenization job. The code calls `\__hook_try_patch_with_catcodes:Nnnnw` with a different catcode setting:

- The current catcode setting;
- Switching the catcode of `@`;
- Switching the expl3 syntax on or off;
- Both of the above.

If patching succeeds, `\__hook_try_patch_with_catcodes:Nnnnw` has the side-effect of patching the macro `#1` (which may be an internal from the command whose name is `#2`).

```
        \tl_set:Ne \l__hook_tmpa_tl
          {
            \int_compare:nNnTF { \char_value_catcode:n {`\@ } } = { 12 }
              { \exp_not:N \makeatletter } { \exp_not:N \makeatother }
          }
```

```
        \tl_set:Ne \l__hook_tmpb_tl
          {
            \bool_if:NTF \l__kernel_expl_bool
              { \ExplSyntaxOff }
              { \ExplSyntaxOn \char_set_catcode_space:n { 32 }}
          }
        \use:e
          {
            \exp_not:N \__hook_try_patch_with_catcodes:Nnnnw
                \exp_not:n { #1 {#2} {#3} }
              { \prg_do_nothing: }
              { \exp_not:V \l__hook_tmpa_tl } % @
              { \exp_not:V \l__hook_tmpb_tl } % _:
              {
                \exp_not:V \l__hook_tmpa_tl   % @
                \exp_not:V \l__hook_tmpb_tl   % _:
              }
          }
              \q_recursion_tail \q_recursion_stop
```

If no catcode setting succeeds, give up and raise an error. The command isn't changed in any way in that case.

```
          {
            \msg_error:nnee { hooks } { cant-patch }
              { \c_backslash_str #2 } { retok }
          }
      }
  }
```

(*End of definition for* `\__hook_retokenize_patch:Nnn`.)

`\__hook_try_patch_with_catcodes:Nnnnw` This function is a simple wrapper around `\__hook_cmd_if_scanable:NnTF` and `\__hook_patch_retokenize:Nnnn` if the former returns ⟨*true*⟩, plus some debug messages.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_try_patch_with_catcodes:Nnnnw}
⟨latexrelease⟩                  {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_try_patch_with_catcodes:Nnnnw #1 #2 #3 #4
  {
    \quark_if_recursion_tail_stop_do:nn {#4} { \use:n }
    \__hook_patch_debug:e { ++~trying~to~patch~by~retokenization }
    \__hook_cmd_if_scanable:NnTF {#1} {#4}
      {
        \__hook_patch_debug:e { ++~macro~can~be~retokenized~cleanly }
        \__hook_patch_debug:e { ==~retokenizing~macro~now }
        \__hook_patch_retokenize:Nnnn #1 { cmd / #2 / #3 } {#3} {#4}
        \use_i_delimit_by_q_recursion_stop:nw \use_none:n
      }
      {
        \__hook_patch_debug:e { --~macro~cannot~be~retokenized~cleanly }
        \__hook_try_patch_with_catcodes:Nnnnw #1 {#2} {#3}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_try_patch_with_catcodes:Nnnnw}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_try_patch_with_catcodes:Nnnnw #1 #2 #3 #4
```

```
⟨latexrelease⟩  {
⟨latexrelease⟩    \quark_if_recursion_tail_stop_do:nn {#4} { \use:n }
⟨latexrelease⟩    \__hook_patch_debug:e { ++~trying~to~patch~by~retokenization }
⟨latexrelease⟩    \__hook_cmd_if_scanable:NnTF {#1} {#4}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_patch_debug:e { ++~macro~can~be~retokenized~cleanly }
⟨latexrelease⟩        \__hook_patch_debug:e { ==~retokenizing~macro~now }
⟨latexrelease⟩        \__hook_patch_retokenize:Nnnn #1 {#2} {#3} {#4}
⟨latexrelease⟩        \use_i_delimit_by_q_recursion_stop:nw \use_none:n
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_patch_debug:e { ---macro~cannot~be~retokenized~cleanly }
⟨latexrelease⟩        \__hook_try_patch_with_catcodes:Nnnnw #1 {#2} {#3}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_patch_with_catcodes:Nnnnw`.)

`\kerneltmpDoNotUse` This is an oddity required to be safe (as safe as reasonably possible) when patching the command. The entirety of

⟨*prefixes*⟩ `\def` ⟨*cs*⟩ ⟨*parameter text*⟩ `{`⟨*replacement text*⟩`}`

will go through `\scantokens`. The ⟨*parameter text*⟩ and ⟨*replacement text*⟩ are what we are trying to retokenize, so not much worry there. The other items, however, should "just work", so some care is needed to not use too fancy catcode settings. Therefore we can't use an expl3-named macro for ⟨*cs*⟩, nor the expl3 versions of `\def` or the ⟨*prefixes*⟩. That is why the definitions that will eventually go into `\scantokens` will use the oddly (but hopefully clearly)-named `\kerneltmpDoNotUse`:

```
\cs_new_eq:NN \kerneltmpDoNotUse !
```

> *PhO: Maybe this can be avoided by running the* ⟨`parameter text`⟩ *and the* ⟨`replacement text`⟩ *separately through* `\scantokens` *and then putting everything together at the end.*

(*End of definition for* `\kerneltmpDoNotUse`.)

`\__hook_patch_required_catcodes:` Here are the catcode settings that are *mandatory* when retokenizing commands. These are the minimum necessary settings to perform the definitions: they identify control sequences, which must be escaped with $_0$, delimit the definition with $\{_1$ and $\}_2$, and mark parameters with $\#_6$. Everything else may be changed, but not these.

```
\cs_new_protected:Npn \__hook_patch_required_catcodes:
  {
    \char_set_catcode_escape:N \\
    \char_set_catcode_group_begin:N \{
    \char_set_catcode_group_end:N \}
    \char_set_catcode_parameter:N \#
    % \int_set:Nn \tex_endlinechar:D { -1 }
    % \int_set:Nn \tex_newlinechar:D { -1 }
  }
```

> *PhO:* etoolbox *sets the* `\endlinechar` *and* `\newlinechar` *when patching, but as far as I tested these didn't make much of a difference, so I left them out for now. Maybe* `\newlinechar=-1` *avoids a space token being added after the definition.*

PhO: If the patching is split by ⟨parameter text⟩ and ⟨replacement text⟩, then only # will have to stay in that list.

PhO: Actually now that we patch \UseHook{cmd/foo/before}, all the tokens there need to have the right catcodes, so this list now includes all lowercase letters, U and H, the slash, and whatever characters in the command name... sigh...

(*End of definition for* `\__hook_patch_required_catcodes:`.)

`\__hook_cmd_if_scanable:NnTF` Here we'll do a quick test if the command being patched can in fact be retokenized with the specific catcode setting without changing in meaning. The test is straightforward:

1. apply `\meaning` to the command;
2. split the ⟨*prefixes*⟩, ⟨*parameter text*⟩ and ⟨*replacement text*⟩ and arrange them as

   ⟨*prefixes*⟩`\def\kerneltmpDoNotUse`⟨*parameter text*⟩`{`⟨*replacement text*⟩`}`
3. rescan that with the given catcode settings, and do the definition; then finally
4. compare `\kerneltmpDoNotUse` with the original command.

If both are `\ifx`-equal, the command can be safely patched.

```
\prg_new_protected_conditional:Npnn \__hook_cmd_if_scanable:Nn #1 #2 { TF }
  {
    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
    \use:e
      {
        \cs_set:Npn \__hook_tmp:w
            ##1 \tl_to_str:n { macro: } ##2 -> ##3 \s__hook_mark
          { ##1 \def \kerneltmpDoNotUse ##2   {##3} }
        \tl_set:Ne \exp_not:N \l__hook_tmpa_tl
          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
      }
    \tl_rescan:nV { #2 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
    \token_if_eq_meaning:NNTF #1 \kerneltmpDoNotUse
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_cmd_if_scanable:NnTF`.)

`\__hook_guess_arg_count:NN` `\__hook_guess_arg_count:wN` `\__hook_guess_arg_count:nw` Looks at the parameter text of a macro, and counts the parameters by looking at the number after a `#`, and checking if they are sequential. This macro assumes that all parameters are marked with hashes, and not other characters, and that there is no "trick parameter".

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_guess_arg_count:NN}
⟨latexrelease⟩                      {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_guess_arg_count:NN #1
  {
    \exp_after:wN \__hook_guess_arg_count:wN
      \token_to_meaning:N #1 \s__hook_mark
  }
\exp_last_unbraced:NNNNo
```

```
\cs_new_protected:Npe \__hook_guess_arg_count:wN
    #1 { \tl_to_str:n { macro: } } #2 \s__hook_mark #3
  {
    \int_set:Nn #3
      {
        \exp_not:N \__hook_guess_arg_count:nw { 0 } #2
          \c_hash_str 0 \s__hook_mark
      }
  }
\use:e
  { \cs_new:Npn \exp_not:N \__hook_guess_arg_count:nw #1 #2 \c_hash_str #3 }
  {
    \int_compare:nNnTF { #1 + 1 } = {#3}
      { \__hook_guess_arg_count:nw {#3} }
      { #1 \__hook_use_none_delimit_by_s_mark:w }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_guess_arg_count:NN}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_guess_arg_count:NN
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_guess_arg_count:NN`*,* `\__hook_guess_arg_count:wN`*, and* `\__hook_guess_arg_count:nw`*.*)

`\__hook_patch_retokenize:Nnnn`

Then, if `\__hook_cmd_if_scanable:NnTF` returned true, we can go on and patch the command.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_patch_retokenize:Nnnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_patch_retokenize:Nnnn #1 #2 #3 #4
  {
```

Here, when patching by retokenization, we can only guess the number of arguments of the macro.

```
    \__hook_guess_arg_count:NN #1 \l__hook_patch_num_args_int
```

Then we redefine the hook to have the right number of arguments. Disabling the hook, undefining the `parameter` token list then calling `\__hook_make_usable:nn` are enough to redefine the hook to the extent we want. Code stored in the hook and other metadata about it are not lost in the process.

```
    \__hook_disable:n {#2}
    \cs_undefine:c { c__hook_#2_parameter_tl }
    \__hook_make_usable:nn {#2} { \l__hook_patch_num_args_int }
    \tl_set:Ne \l__hook_tmpa_tl
      { \exp_args:Ne \tl_to_str:n { \__hook_braced_parameter:n {#2} } }
    \use:e
      {
        \str_replace_all:Nnn \exp_not:N \l__hook_tmpa_tl
          { ## } { \c_hash_str }
      }
```

Then, make some things `\relax` to avoid lots of `\noexpand` below.

```
    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
    \use:e
      {
```

Now we'll define `\__hook_tmp:w` such that it splits the `\meaning` of the macro (`#1`) into its three parts:

`####1.` ⟨*prefixes*⟩

`####2.` ⟨*parameter text*⟩

`####3.` ⟨*replacement text*⟩

and arrange that a complete definition, then place the `before` or `after` hooks around the ⟨*replacement text*⟩: accordingly.

```
      \cs_set:Npn \__hook_tmp:w
          ##1 \tl_to_str:n { macro: } ##2 -> ##3 \s__hook_mark
        {
          ##1 \def \kerneltmpDoNotUse ##2
            {
              \str_if_eq:nnT {#3} { before }
                {
                  \token_to_str:N \UseHookWithArguments {#2}
                    { \int_use:N \l__hook_patch_num_args_int }
                    \l__hook_tmpa_tl
                }
              ##3
              \str_if_eq:nnT {#3} { after }
                {
                  \token_to_str:N \UseHookWithArguments {#2}
                    { \int_use:N \l__hook_patch_num_args_int }
                    \l__hook_tmpa_tl
                }
            }
        }
```

Now we just have to get the `\meaning` of the command being patched and pass it through the meat grinder above.

```
        \tl_set:Ne \exp_not:N \l__hook_tmpa_tl
          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
      }
```

Now rescan with the given catcode settings (overridden by the `\__hook_patch_required_catcodes:`), and implicitly (by using the rescanned token list) carry out the definition from above.

```
    \tl_rescan:nV { #4 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
```

And to close, copy the newly-defined command into the old name and the patching is finally completed:

```
    \cs_gset_eq:NN #1 \kerneltmpDoNotUse
```

Finally, update the hook code.

```
    \__hook_update_hook_code:n {#2}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_patch_retokenize:Nnnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_patch_retokenize:Nnnn #1 #2 #3 #4
⟨latexrelease⟩  {
```

```
⟨latexrelease⟩    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
⟨latexrelease⟩    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
⟨latexrelease⟩    \use:x
⟨latexrelease⟩      {
⟨latexrelease⟩        \cs_set:Npn \__hook_tmp:w
⟨latexrelease⟩            ####1 \tl_to_str:n { macro: } ####2 -> ####3 \s__hook_mark
⟨latexrelease⟩          {
⟨latexrelease⟩            ####1 \def \kerneltmpDoNotUse ####2
⟨latexrelease⟩              {
⟨latexrelease⟩                \str_if_eq:nnT {#3} { before }
⟨latexrelease⟩                  { \token_to_str:N \UseHook { cmd / #2 / #3 } }
⟨latexrelease⟩                ####3
⟨latexrelease⟩                \str_if_eq:nnT {#3} { after }
⟨latexrelease⟩                  { \token_to_str:N \UseHook { cmd / #2 / #3 } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \tl_set:Nx \exp_not:N \l__hook_tmpa_tl
⟨latexrelease⟩          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
⟨latexrelease⟩      }
⟨latexrelease⟩    \tl_rescan:nV { #4 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
⟨latexrelease⟩    \cs_gset_eq:NN #1 \kerneltmpDoNotUse
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_patch_retokenize:Nnnn`.)

4.5 Messages

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{wrong-cmd-hook}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{wrong-cmd-hook}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\msg_new:nnnn { hooks } { wrong-cmd-hook }
⟨latexrelease⟩  {
⟨latexrelease⟩    Generic~hook~‘cmd/#1/#2’~is~invalid.
⟨latexrelease⟩%    The~hook~should~be~‘cmd/#1/before’~or~‘cmd/#1/after’.
⟨latexrelease⟩  }
⟨latexrelease⟩  {
⟨latexrelease⟩    You~tried~to~add~a~generic~hook~to~command~\iow_char:N \\#1,~but~‘#2’~
⟨latexrelease⟩    is~an~invalid~component.~Only~‘before’~or~‘after’~are~allowed.
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
\msg_new:nnnn { hooks } { cant-patch }
  {
    Generic~hooks~cannot~be~added~to~’#1’.
  }
  {
    You~tried~to~add~a~hook~to~’#1’,~but~LaTeX~was~unable~to~
    patch~the~command~because~it~\__hook_unpatchable_cases:n {#2}.
  }
\cs_new:Npn \__hook_unpatchable_cases:n #1
  {
    \str_case:nn {#1}
```

```
      {
        { undef } { doesn't~exist }
        { macro } { is~not~a~macro }
        { expl3 } { is~a~private~expl3~macro }
        { retok } { can't~be~retokenized~cleanly }
      }
  }
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltcmdhooks}%
⟨latexrelease⟩                  {The~hook~management~system~for~commands}
⟨latexrelease⟩
```

The command `\__hook_cmd_begindocument_code:` is used in an internal hook, so we need to make sure it has a harmless definition after rollback as that will not remove it from the kernel hook.

```
⟨latexrelease⟩\cs_set_eq:NN \__hook_cmd_begindocument_code: \prg_do_nothing:
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨@@=⟩
```

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

\# 480
\/ 218, 293
\\ 477, 615
\{ 478
\} 479

A

\AddToHook *4*
\AddToHookWithArguments *3*

B

\begin 82
begindocument (hook) *2*
bool commands:
  \bool_if:NTF 178, 182, 194, 253, 257, 269, 413
  \c_false_bool 157, 167, 403, *12*
  \c_true_bool 162, *12*

C

\caption *3*
char commands:
  \char_set_catcode_escape:N 477
  \char_set_catcode_group_begin:N 478
  \char_set_catcode_group_end:N . . 479
  \char_set_catcode_parameter:N . . 480
  \char_set_catcode_space:n 415
  \char_value_catcode:n 408
cmd (hook) *5*, *6*
cmd/#1/after (hook) *17*
cmd/#1/before (hook) *17*
cmd/⟨*name*⟩/after (hook) *2*, *4*
cmd/⟨*name*⟩/before (hook) *2*, *2*
cmd/foo/before (hook) *4*
cmd/section/after (hook) *4*, *5*
cmd/section/before (hook) *4*
cs commands:
  \cs:w 33, *9*
  \cs_end: 33
  \cs_gset:Npn 383
  \cs_gset_eq:NN 72, 573, 601
  \cs_gset_protected:Npn . 247, 457, 579
  \cs_if_exist:NTF 89, 97
  \cs_new:Npn 319, 329, 339, 341, 343, 354, 375, 394, 395, 398, 519, 627
  \cs_new_eq:NN 474, 13, 14
  \cs_new_protected:Npe 509
  \cs_new_protected:Npn 18, 22, 24, 39, 41, 50, 52, 62,

70, 79, 93, 113, 133, 146, 151, 160, 165, 172, 312, 400, 438, 475, 503, 532
\cs_parameter_spec:N *14*
\cs_prefix_spec:N 223, 298
\cs_set:Npn ... 215, 290, 490, 549, 585
\cs_set_eq:NN 187, 188, 208, 209, 214, 262, 263, 283, 284, 289, 486, 487, 545, 546, 581, 582, 640
\cs_set_protected:Npn 189, 264
\cs_to_str:N 128, 149, 163, 168
\cs_undefine:N 75, 226, 528, 536

D

\DebugHooksOn *3*
\def 492, 552, 588
\DisableGenericHook *5*

E

else commands:
\else: 363, 387
\EndIncludeInRelease 36, 47, 61, 244, 311, 454, 473, 525, 529, 576, 603, 606, 618
\EndModuleRelease 642
exp commands:
\exp_after:wN 29, 31, 186, 261, 315, 316, 378, 386, 389, 505
\exp_args:Nc 45, 58, 90, 148
\exp_args:Ne 539
\exp_args:Nf 177, 252
\exp_args:NNc 162, 167
\exp_args:NNo 185, 260
\exp_args:NNV 204, 279
\exp_args:No 185, 204, 260, 279
\exp_args:NV 205, 280
\exp_last_unbraced:Nf 126
\exp_last_unbraced:NNNNo ... 132, 508
\exp_last_unbraced:NNo 397
\exp_not:N 124, 126, 127, 128, 200, 215, 222, 223, 230, 235, 275, 290, 297, 298, 302, 306, 323, 372, 377, 385, 409, 419, 493, 494, 514, 519, 542, 569, 570, 597, 598, *15*
\exp_not:n 187, 200, 203, 205, 240, 262, 275, 278, 280, 308, 370, 420, 422, 423, 425, 426, *15*
\ExplSyntaxOff 414, 643
\ExplSyntaxOn 415, 3, *3*

F

fi commands:
\fi: 358, 360, 361, 365, 379, 390, 391, 394, *19*
\foo *13*

G

group commands:
\group_begin: 28, 217, 292
\group_end: 30, 221, 296, *9*

H

hook commands:
\g_hook_patch_action_list_tl ... 107, 140, 6
hook internal commands:
__hook_braced_parameter:n . 237, 539
__hook_cmd_begindocument_code: . 62, 70, 75, 78, 640, *27*
__hook_cmd_if_scanable:Nn 484
__hook_cmd_if_scanable:NnTF ... 442, 461, 484, *21*
__hook_cmd_patch_xparse:Nnn ... 144, 165, 165
__hook_cmd_try_patch:nn .. 68, 79, 79
__hook_debug:n 19, 27, 44, 55, 64, 65, 81, 85
__hook_def_cmd:w 208, 214, 283, 289, 315, 13, 14, *17*
\g__hook_delayed_patches_prop ... 17, 67, 73, 74
__hook_disable:n 225, 535
__hook_double_hashes:n 204, 279, 341, 341, 396, *18*
__hook_double_hashes:w 341, 342, 343, 373, 396, 399, *18*
__hook_double_hashes_group:n ... 341, 349, 395
__hook_double_hashes_output:N .. 341, 346, 354, *19*
__hook_double_hashes_output_aux:N 341, 368, 375, 383
__hook_double_hashes_space:w ... 341, 350, 398
__hook_double_hashes_stop:w ... 341, 357, 394
__hook_exp_not:n 187, 188, 192, 203, 262, 263, 267, 278, *15*
__hook_exp_not:NN 209, 215, 235, 240, 284, 290, 306, 308, 13, 13
__hook_guess_arg_count:NN 501, 501, 503, 526, 528, 534
__hook_guess_arg_count:nw 501, 514, 519, 522
__hook_guess_arg_count:wN 501, 505, 509
\c__hook_hash_tl 195, 196, 198, 270, 271, 273, 360, 11
\c__hook_hashes_tl 361, 11, *19*
__hook_if_declared:nTF 83

__hook_if_has_hash:n 327
__hook_if_has_hash:nTF 185, 260, 327, *14*
__hook_if_has_hash:w 327, 328, 329, 335, 337
__hook_if_has_hash_check:w 327, 334, 339
__hook_if_has_hash_p:n 327
__hook_if_public_command:N . . . 124
__hook_if_public_command:NTF 93, 103, *11*
__hook_if_public_command:w 93, 127, 133
__hook_make_prefixes:w 170, 223, 298, 319, 324
__hook_make_usable:nn . . 227, 537, *15*
\l__hook_param_text_tl 191, 212, 266, 287, 316, 8, *17*
__hook_patch_check:NNnn 93, 97, 100, 103, 113
__hook_patch_cmd_or_delay:Nnn 32, 45, 58, 62, 62, 72, *9*
__hook_patch_command:Nnn 72, 90, 93, 93, *10*
__hook_patch_debug:n . . . 18, 18, 95, 96, 99, 102, 105, 174, 249, 405, 441, 444, 445, 450, 460, 463, 464, 469
__hook_patch_DeclareRobustCommand:Nnn 142, 146, 146, *12*
__hook_patch_DeclareRobustCommand_aux:Nnn 148, 151
__hook_patch_expand_redefine:NNnn 157, 162, 167, 170, 170, 172, 245, 247, 403, *20*
__hook_patch_newcommand:Nnn 143, 156, 160, 160, *12*
\l__hook_patch_num_args_int 175, 180, 183, 197, 227, 236, 250, 255, 258, 272, 534, 537, 557, 564, 7
\l__hook_patch_prefixes_tl 222, 297, 314, 8, *17*
__hook_patch_required_catcodes: 475, 475, 496, 572, 600, *25*
__hook_patch_retokenize:Nnnn . . . 446, 465, 530, 530, 532, 577, 579, *21*
__hook_redefine_with_hooks:Nnnn 170, 230, 302, 312, *15*
\l__hook_replace_text_tl . . . 192, 199, 200, 201, 206, 213, 240, 267, 274, 275, 276, 281, 288, 308, 8, *14*
__hook_retokenize_patch:Nnn 108, 400, 400
__hook_tmp:w 189, 195, 196, 198, 264, 270, 271, 273, 487, 490, 494, 546, 549, 570, 582, 585, 598, *25*
\l__hook_tmpa_tl 182, 184, 186, 257, 259, 261, 406, 422, 425, 493, 496, 538, 542, 558, 565, 569, 572, 597, 600
\l__hook_tmpb_tl 411, 423, 426
__hook_try_patch_with_catcodes:Nnnnw 419, 436, 436, 438, 451, 455, 457, 470, *20*
__hook_try_put_cmd_hook:n 20, 20, 22, 37, 39, 48, 50
__hook_try_put_cmd_hook:w 20, 23, 24, 40, 41, 51, 52
__hook_unpatchable_cases:n 625, 627
__hook_update_hook_code:n . 242, 574
__hook_use_none_delimit_by_s_mark:w 523
Hooks:
begindocument *2*
cmd *5*, *6*
cmd/#1/after *17*
cmd/#1/before *17*
cmd/⟨*name*⟩/after *2*, *4*
cmd/⟨*name*⟩/before *2*, *2*
cmd/foo/before *4*
cmd/section/after *4*, *5*
cmd/section/before *4*

I

if commands:
\if:w 359
\if_catcode:w 377, 385, *19*
\if_meaning:w 356, 360, 361, 388
\ifx *18*
\IncludeInRelease 20, 37, 48, 170, 245, 436, 455, 501, 526, 530, 577, 604, 607, 637
int commands:
\int_compare:nNnTF . 180, 255, 408, 521
\int_new:N 7
\int_set:Nn 175, 218, 250, 293, 481, 482, 512
\int_step_inline:nnn 183, 197, 258, 272
\int_use:N 236, 557, 564
\c_zero_int 180, 255
iow commands:
\iow_char:N 615
\iow_term:n 19, 27, 44, 55, 64, 66, 82, 86

K

kernel internal commands:
__kernel_cmd_if_xparse:NTF . 144, *11*

__kernel_cs_parameter_spec:N ... 177, 252, 402
\l__kernel_expl_bool ... 413
\kerneltmpDoNotUse ... 474, 486, 492, 497, 545, 552, 573, 581, 588, 601, *23*

M

\makeatletter ... 409
\makeatother ... 409
msg commands:
\msg_error:nnnn ... 59, 117, 431
\msg_new:nnnn ... 609, 619

N

\newcommand ... *3*
\NewCommandCopy ... *4*
\NewDocumentCommand ... *3*
\NewHook ... *6*
\NewHookPair ... *4*
\NewHookWithArguments ... *5*
\NewModuleRelease ... 4
\NewReversedHookWithArguments ... *5*
\notexpanded ... *18*

P

prg commands:
\prg_do_nothing: ... 209, 284, 421, 640
\prg_new_conditional:Npnn ... 327
\prg_new_protected_conditional:Npnn ... 123, 484
\prg_return_false: ... 138, 340, 499
\prg_return_true: ... 137, 340, 498
prop commands:
\prop_gclear:N ... 74
\prop_gput:Nnn ... 67
\prop_map_function:NN ... 73
\prop_new:N ... 17

Q

quark commands:
\quark_if_recursion_tail_stop_do:nn ... 440, 459
\quark_new:N ... 15, 16
\q_recursion_stop ... 429
\q_recursion_tail ... 429
quark internal commands:
\q__hook_recursion_stop ... 15, 342, 343, 352, 394
\q__hook_recursion_tail ... 15, 342, 356

R

\refstepcounter ... *6*
\relax ... *9*

S

scan commands:
\scan_stop: 486, 487, 545, 546, 581, 582
scan internal commands:
\s__hook_mark ... 23, 25, 40, 42, 51, 53, 129, 134, 328, 339, 491, 494, 506, 510, 515, 550, 570, 586, 598
\section ... *3*
str commands:
\c_backslash_str ... 163, 432
\c_hash_str ... 515, 519, 543
\str_case:nn ... 629
\str_case:nnTF ... 56
\str_count:n ... 177, 252
\str_if_eq:nnTF ... 231, 303, 402, 554, 561, 590, 593
\str_replace_all:Nnn ... 542
\string ... 82
sys commands:
\sys_if_engine_luatex:TF ... 381

T

TeX and LaTeX 2ε commands:
\@ ... 408
\@if@DeclareRobustCommand ... 142
\@if@newcommand ... 143, 154, *12*
\@kernel@after@begindocument ... 77
\AddToHook ... *2*
\AddToHookNext ... *2*
\AddToHookNextWithArguments ... *2*
\AddToHookWithArguments ... *2*
\apptocmd ... *2*
\DeclareRobustCommand ... *3*
\def ... *22*
\endlinechar ... *22*
\escapechar ... *15*
\g@addto@macro ... 77
\ifx ... *23*
\meaning ... *23*
\newcommand ... *3*
\NewCommandCopy ... *11*
\NewDocumentCommand ... *3*
\newlinechar ... *22*
\noexpand ... *24*
\patchcmd ... *3*
\pretocmd ... *2*
\relax ... *24*
\robust@command@act ... 106, *10*
\robust@command@chk@safe ... 153, *12*
\scantokens ... *20*
\section ... *5*
\ShowCommand ... *11*
tex commands:
\tex_alignmark:D ... 388

\tex_endlinechar:D 481
\tex_escapechar:D 218, 293
\tex_gdef:D 208, 283, *17*
\tex_newlinechar:D 482
\tex_xdef:D 214, 289, *17*
tl commands:
\c_space_tl 398
\tl_clear:N 212, 287
\tl_const:Nn 11, 12
\tl_gset:Nn 140
\tl_if_empty:nTF 136, 321, 340
\tl_if_head_is_group:nTF 348
\tl_if_head_is_N_type:nTF 345
\tl_if_single_token:nTF 331
\tl_new:N 6, 8, 9, 10
\tl_put_right:Nn . . . 184, 198, 259, 273
\tl_rescan:nn 496, 572, 600
\tl_set:Nn 182, 195, 196, 199, 201, 222, 257, 270, 271, 274, 276, 297, 406, 411, 493, 538, 569, 597
\tl_set_eq:NN 213, 288
\tl_to_str:n 129, 134, 491, 510, 539, 550, 586
\tl_trim_spaces:n 323, *17*
token commands:
\token_if_eq_catcode:NNTF 333
\token_if_eq_meaning:NNTF 203, 278, 497
\token_if_macro:NTF 100, *11*
\token_to_meaning:N 96, 494, 506, 570, 598
\token_to_str:N 95, 96, 118, 556, 563, 591, 594

U

\unexpanded *13*
use commands:
\use:n 121, 188, 191, 219, 228, 263, 266, 294, 300, 366, 417, 440, 459, 488, 518, 540, 547, 583, *19*
\use:nn 233, 305
\use_i:nnnn 364, *19*
\use_i_delimit_by_q_recursion_stop:nw 447, 466
\use_ii:nn 73, 155
\use_ii:nnnn 378, 386, 389
\use_ii_i:nn 232, 304
\use_none:n 369, 447, 466
\UseHook 306, 591, 594, *23*
\UseHookWithArguments . . 235, 556, 563, *6*